

CNN-MalGen06

Beginner's Code Guide

A block-by-block walkthrough of data.py, model.py, and train.py

Written for a reader who understands matrix math but is new to Python and machine learning code

Repository revision reviewed: master branch, July 31, 2026

<https://github.com/PaladinMG/CNN-MalGen06>

How to use this guide

This document follows the program in the order Python encounters it. It does not repeat every tensor formula, because the intended reader already understands matrix operations. Instead, it explains what the Python syntax means, why each block exists, and how data moves between blocks.

Line numbers: The repository may change after this document is generated. The line ranges are anchors for the July 31, 2026 master revision, not permanent identifiers.

Contents

1. Python patterns used throughout the project
2. Project execution map
3. model.py - how the neural network is assembled
4. data.py - how images, masks, crops, augmentation, and caches are prepared
5. train.py - how losses, optimization, validation, compilation, and saving work
6. Utility scripts and cluster files
7. Reading error messages and making safe changes
8. Glossary

1. Python patterns used throughout the project

Indentation defines structure

Python does not use braces to mark the inside of a function, loop, or if statement. Indented lines belong to the block above them. Returning to the previous indentation level ends the block.

```
if device.type == "cuda":
    enable_gpu_options()
    print("GPU enabled")
print("This line runs either way")
```

Functions

A line beginning with `def` creates a reusable operation. Text inside parentheses names the inputs. The arrow after the parentheses is a type hint describing the expected output; it does not perform the conversion.

```
def coarse_targets(mask: torch.Tensor) -> torch.Tensor:
    mapping = torch.tensor(COARSE_MAPPING, device=mask.device)
    return mapping[mask]
```

- ``mask: torch.Tensor`` says the input should be a PyTorch tensor.
- ``-> torch.Tensor`` says the function returns a tensor.
- ``return`` sends a value back to the caller and immediately ends the function.

Classes and self

A class is a blueprint for an object that stores both data and functions. ``self`` means “this particular object.” When the dataset stores ``self.crop_size``, every later method can read the same crop size.

```
class SegmentationDataset(Dataset):
    def __init__(self, crop_size: int = 512):
        self.crop_size = crop_size

    def __len__(self) -> int:
        return len(self.samples) * self.patches_per_image
```

Lists, tuples, dictionaries, and indexing

- A list uses square brackets and can be changed: `[image1, image2]`.
- A tuple uses parentheses and is normally treated as fixed: `(0, 1, 2, 1, 3, 4)`.
- A dictionary stores named values: `{"local": tensor, "mask": mask}`.
- `items[0]` gets the first element. Python starts counting at zero.
- `tensor[:, 0:1]` means all batches, channels zero through one, excluding one.
- `None` means “no value was supplied.” Many optional paths test `is not None`.

Loops and conditionals

```
for image_path in sorted(image_dir.iterdir()):
    if image_path.suffix.lower() not in IMAGE_SUFFIXES:
        continue
    samples.append(image_path)
```

The loop visits every file. The `if` rejects unsupported extensions. `continue` skips the rest of the current loop iteration and moves to the next file.

Context managers

`with Image.open(...) as image\_file:` opens a file and guarantees that it is closed when the indented block ends, even if an exception occurs.

Decorators

A line such as `@torch.no\_grad()` changes how the following function runs. Here it tells PyTorch not to construct a gradient graph, because feature-cache creation is preprocessing rather than training.

Type hints with vertical bars

`torch.Tensor | None` means the value may be a tensor or may be absent. `list[int] | None` means either a list of integers or no list.

The main guard

```
if __name__ == "__main__":
    main()
```

This calls `main()` when the file is run directly. It does not call `main()` when another file imports functions from it.

2. Project execution map

The three central files have different responsibilities. Keeping those responsibilities separate makes the project easier to test and optimize.

File	Main responsibility	Typical objects produced
data.py	Find image/mask pairs, choose crop centers, build crops, augment data, and optionally cache source data in VRAM.	Dataset samples and training batches
model.py	Define fixed structural features, encoders, decoder, prediction heads, and final six-class probabilities.	AccurateTissueNet and output tensors
train.py	Split data, compute losses and metrics, run epochs, update weights, schedule learning rates, and save checkpoints.	best_model.pt and last_model.pt

```
train.py
├─ creates SegmentationDataset objects from data.py
├─ creates AccurateTissueNet from model.py
├─ asks the DataLoader for batches
├─ sends batches through the model
├─ calculates loss and gradients
├─ calls optimizer.step() to change weights
├─ validates after each epoch
└─ saves last_model.pt and, when improved, best_model.pt
```

3. model.py - assembling the neural network

`model.py` describes what happens after a batch is ready. It defines the model's parts in `\_\_init\_\_`, then defines the path data follows in `forward`.

Imports and class names

model.py lines 1-30

The file imports PyTorch, neural-network layers, functional operations, and pretrained ResNet builders.

`CLASS\_NAMES` fixes the final class order. Every mask, class weight, probability channel, and output label must use this exact order.

`COARSE\_CLASS\_NAMES` describes the five parent classes used by the first prediction head. Fibrocartilage and Muscle share a parent at this stage.

```
CLASS_NAMES = (
    "Bone", "Fibrocartilage", "Cartilage",
    "Muscle", "Marrow", "Background",
)
```

Important: Changing this order without converting masks and checkpoints would silently change the meaning of output channels.

ModelConfig dataclass

model.py lines 32-37

A dataclass is a compact container for configuration values. ``frozen=True`` prevents accidental changes after creation.

The model stores context scale, whether pretrained weights were requested, and decoder width. Checkpoints can later recreate the same architecture from this configuration.

```
@dataclass(frozen=True)
class ModelConfig:
    context_scale: int = 4
    pretrained: bool = True
    decoder_channels: int = 64
```

The _kernel helper

model.py lines 39-40

This function converts a nested Python list into a 3×3 tensor, reshapes it to the four-dimensional layout expected by ``conv2d``, and divides by a normalization constant.

It is used only while constructing fixed image-processing filters.

HandcraftedFeatures.__init__

model.py lines 43-100

This class calculates nine structural maps before the learned feature stem. The filters themselves are constants, not trainable weights.

``register_buffer`` attaches a tensor to the module so it moves to the GPU with the model and appears in device management, but the optimizer does not update it.

The Sobel filters estimate first derivatives. Laplace estimates local curvature. Dxx, Dyy, and Dxy estimate second derivatives. The Gaussian kernel smooths local moments.

The code combines several filters into ``derivative_kernels`` and repeats the Gaussian kernel into ``gaussian_kernels``. This reduces many tiny convolution launches to two larger calls.

Python detail: ``persistent=False`` means the fused banks do not become permanent checkpoint entries. The original named buffers remain compatible with older checkpoints.

HandcraftedFeatures.forward

model.py lines 101-164

The method first verifies the input has four dimensions and three color channels.

It converts RGB to one luminance channel. One fused convolution produces six derivative images. A grouped Gaussian convolution smooths five moment images independently.

The remaining lines combine those maps into gradient magnitude, local deviation, structure-tensor eigenvalues and coherence, Hessian determinant, and Hessian eigenvalues.

The nine maps are concatenated along the channel dimension. `asinh`, fixed scales, and clipping keep extreme values numerically manageable without normalizing each tile separately.

```
if rgb.ndim != 4 or rgb.shape[1] != 3:
    raise ValueError(...)
...
return torch.asinh(raw / self.scales).clamp(-8, 8)
```

`_group_count`

model.py lines 166-170

GroupNorm needs a number of groups that evenly divides the channel count.

The loop tries 16 groups, then 8, 4, 2, and 1. The first divisor is returned. The final `return 1` is a safety fallback.

```
for groups in (16, 8, 4, 2, 1):
    if channels % groups == 0:
        return groups
```

ConvBlock

model.py lines 172-190

A ConvBlock contains two convolution → normalization → GELU sequences.

The `if` inside the residual definition decides whether the input needs a 1×1 projection to match the output channel count. If the counts already match, `nn.Identity()` returns the input unchanged.

The forward method adds the transformed path to the residual path. This is a residual connection.

DecoderBlock

model.py lines 193-205

A DecoderBlock enlarges a low-resolution feature map to the size of a skip connection using bilinear interpolation.

A 1×1 projection changes channel count. `torch.cat` joins the enlarged features and encoder skip features. ConvBlock then refines the combined tensor.

`AccurateTissueNet.__init__`: input encoders

model.py lines 208-250

The constructor rejects context scales below one, stores the configuration, then builds two pretrained networks.

ResNet-50 is used for the local high-detail patch. ResNet-18 is used for the wider context image.

Instead of keeping each whole ResNet object, the code saves the individual layers under descriptive names. This makes the custom fusion and decoder easier to write.

Machine-learning idea: Pretrained weights provide generic visual filters learned from ImageNet. Training then adapts them to tissue images.

`AccurateTissueNet.__init__`: feature fusion and decoder

model.py lines 251-279

The nine handcrafted maps pass through a small learned stem and are fused with the RGB stem.

The deepest ResNet-50 output is projected from 2,048 channels to 512. The ResNet-18 context output is projected from 512 to 256.

``context_film`` converts a global context vector into gamma and beta values that modulate the local bottleneck.

Four DecoderBlocks progressively restore spatial resolution using skip connections. Three pixel-level heads predict coarse class, conditional Muscle, and Bone/Fibro boundary. A separate regional head predicts whether Muscle exists in the patch region.

Normalization and center-context helper methods

model.py middle helper methods

The RGB normalization method subtracts ImageNet means and divides by ImageNet standard deviations before pretrained encoders see the images.

The center-context helper extracts the central portion of the context feature map that corresponds most closely to the local field of view, then resizes it to match the local bottleneck.

AccurateTissueNet.forward: validation and local path

model.py lines 318-348

The first ``if`` confirms local and context batches have identical tensor shapes after context resizing.

The local RGB image is normalized and sent through the ResNet-50 stem.

If handcrafted features were not supplied by the data cache, the model calculates them now. The ``elif`` validates a supplied feature tensor.

RGB and structural stems are concatenated and fused, then passed through the four ResNet-50 stages. The deepest features are projected to the local bottleneck width.

```
if handcrafted_features is None:
    handcrafted_features = self.handcrafted(local_rgb)
elif invalid_shape:
    raise ValueError(...)
```

AccurateTissueNet.forward: context and FiLM

model.py lines 349-368

The context image passes through ResNet-18. Adaptive average pooling reduces the whole context map to one vector per sample.

``chunk(2, dim=1)`` splits the context network output into gamma and beta halves. Gamma scales local channels and beta shifts them.

The center portion of the context map is also projected and concatenated with the modulated local bottleneck. This gives the decoder both broad regional information and spatially centered context features.

AccurateTissueNet.forward: decoder and heads

model.py lines 369-393

Four decoder calls move from deepest features toward full resolution while adding local skip connections.

A final interpolation exactly matches the input height and width.

The regional Muscle-presence logit is calculated from pooled local and context vectors. It is added, after multiplication by a learned scale, to the pixel-level Muscle logits.

The method returns a dictionary of raw logits. It does not yet return final six-class probabilities.

probabilities_from_outputs

model.py lines 394-412

Softmax converts the five coarse logits into probabilities that sum to one. Sigmoid converts the conditional Muscle logit into the probability of Muscle given the shared parent class.

The parent probability is split into Fibrocartilage and Muscle. The final concatenation restores the six-class order used by masks.

```
parent = coarse[:, 1:2]
return torch.cat([
    coarse[:, 0:1],
    parent * (1 - muscle_given_parent),
    coarse[:, 2:3],
    parent * muscle_given_parent,
    coarse[:, 3:4],
    coarse[:, 4:5],
], dim=1)
```

The model.py self-test

model.py lines 419-429

When ``python model.py`` is run directly, this block creates a model without downloading pretrained weights, generates random local/context inputs, runs inference, counts parameters, and prints the output shape and probability sum.

This is a lightweight sanity check, not a training run.

4. data.py - building training samples

``data.py`` turns large source images and label masks into the smaller aligned tensors used by the model. It also contains the optional VRAM cache.

Imports, constants, and Image.MAX_IMAGE_PIXELS

data.py lines 1-21

The file imports NumPy for CPU mask operations, PyTorch for tensor operations, Pillow for image loading, and torchvision for tensor resizing.

``Image.MAX_IMAGE_PIXELS = None`` disables Pillow's decompression-bomb warning because microscopy images can be very large. This should only be used with trusted source files.

``BACKGROUND_ID = 5`` matches the class order in `model.py`.

discover_samples

data.py lines 23-40

The function converts the supplied root into a Path and expects ``images/`` and ``masks/`` subfolders.

The ``for`` loop visits image files in sorted order. Unsupported suffixes are skipped. The expected mask name is built from the image stem plus ``png``.

Missing masks cause an immediate error instead of allowing a later image/mask mismatch. Every valid pair is appended as a tuple.

After the loop, an empty list also raises an error.

`_neighbor_any` and `bone_fibro_boundary`

data.py lines 42-59

`_neighbor_any` pads a Boolean mask and checks all eight neighboring offsets with two nested loops. The center offset is skipped.

`bone_fibro_boundary` creates Boolean masks for Bone and Fibrocartilage, finds pixels where one touches the other, and optionally expands the boundary in a loop.

```
for dy in range(3):
    for dx in range(3):
        if dy == 1 and dx == 1:
            continue
        result |= padded[dy:dy+height, dx:dx+width]
```

`_extract_centered` for NumPy arrays

data.py lines 61-90

This helper computes a square crop around a center coordinate.

If the requested crop extends outside the image, it calculates how much padding is needed on each side.

For ordinary images, `mode="edge"` repeats edge pixels. For masks, a supplied constant value fills outside pixels with Background.

`np.ascontiguousarray` ensures the result has a simple memory layout before conversion to a tensor.

`_extract_centered_tensor` for PyTorch tensors

data.py lines 91-133

This is the GPU/CPU tensor equivalent of the NumPy crop helper. It assumes channel-first layout `[C, H, W]`.

In-bounds crops remain views into the original tensor. That avoids allocating a new tensor for each sample. Padding is only performed when required.

`_color_jitter_tensor`

data.py lines 135-170

This function is used in the non-cached path. It converts byte RGB values to floating-point values between zero and one, then applies brightness, contrast, saturation, and gamma changes.

The same randomly chosen parameters are used for the local and context images so they remain visually consistent.

Cache-size estimators

data.py lines 172-213

`estimate_rgb_cache_bytes` totals three bytes per source pixel for uint8 RGB.

`estimate_training_cache_bytes` loops through image dimensions and separately estimates RGB, context pyramid, optional nine-channel FP16 features, and two uint8 target maps.

The returned dictionary is printed by `train.py` before allocation.

`_precompute_handcrafted_features`

data.py lines 215-252

The decorator disables gradient tracking. A full nine-channel FP16 output tensor is allocated in VRAM.

Two nested loops move through the large source image in tiles. Each tile is expanded by a two-pixel halo so 3×3 filters near tile edges see neighboring pixels.

The feature extractor processes the expanded tile. Only the central non-halo region is copied into the final output.

Why tile?: Computing nine full-size floating-point maps at once can require a large temporary allocation. Tiling bounds temporary VRAM without changing the final cache size.

`SegmentationDataset.__init__`: parameters and validation

data.py lines 253-297

The constructor stores all dataset settings on ``self``.

The first ``if`` statements reject crop sizes that are not positive multiples of 32 and context scales below one.

Feature caching is rejected unless a device cache exists, because the feature cache is designed for CUDA VRAM.

``sample_indices`` optionally selects only part of the discovered image list, such as an image-level training split.

`SegmentationDataset.__init__`: cache containers

data.py cache setup block

When caching is disabled, cache attributes remain ``None``. When enabled, the constructor creates lists for source images, context pyramid images, CPU masks, device masks, boundaries, sampling coordinates, present classes, and optional handcrafted features.

A HandcraftedFeatures extractor is created only when feature caching is requested.

`SegmentationDataset.__init__`: image-cache loop

data.py cache loop, approximately lines 310-407

The ``for`` loop opens each image and mask once. It validates that the mask is two-dimensional, dimensions match, and class IDs do not exceed five.

RGB is converted from NumPy ``[H, W, C]`` to PyTorch ``[C, H, W]`` and transferred to the cache device.

A reduced context pyramid is built once with bicubic interpolation. A CPU mask remains available for center selection, while device copies of mask and boundary avoid per-patch transfers.

Boundary coordinates and present classes are precomputed to avoid repeatedly scanning an entire 4096×4096 mask.

If feature caching is enabled, the loop prints progress and calculates the nine full-image feature maps.

`__len__`

data.py lines 408-409

The dataset length equals the number of source images multiplied by the requested patches per image.

A DataLoader treats each patch as one sample even though multiple patches may come from the same source image.

`_select_center`: deterministic path*data.py lines 410-435*

If fixed patch centers are required, or augmentation is off, the method creates a near-square grid adapted to the image aspect ratio.

``divmod`` converts the patch number into a row and column. The final centers are evenly distributed across the image.

This deterministic path is required before a patch-level train/validation split so a patch ID always refers to the same location.

`_select_center`: randomized training paths*data.py lines 436 onward*

A random number chooses among boundary-focused, class-focused, and unbiased sampling.

The boundary branch uses cached coordinates when available. Otherwise it calculates them from the mask. A small random jitter prevents all patches from being centered on exactly the same boundary pixels.

The class-focused branch chooses one present foreground class, finds its pixels, and selects one coordinate. If the branch cannot find a coordinate, the method eventually falls back to a random point.

`__getitem__`: convert dataset index into image and patch*data.py beginning of `__getitem__`*

``divmod(index, patches_per_image)`` returns both the source-image index and the patch number within that image.

The following ``if`` chooses between the cached path and the disk-loading path. Both paths eventually produce a channel-first image tensor and a uint8 mask.

`__getitem__`: validation and crop creation*data.py approximately lines 532-620*

The uncached path validates dimensions and mask IDs each time. The cached path already performed those checks during construction.

The selected center is used to extract local RGB and local mask crops.

If a context pyramid exists, the center coordinate is scaled into pyramid coordinates and a 512-pixel crop is taken directly. Otherwise a much larger native context crop is taken and resized.

Optional handcrafted and full-boundary caches are cropped at the same center.

`__getitem__`: augmentation branches*data.py lines 622-674 and following*

Random horizontal flip, vertical flip, and quarter-turn rotation are selected.

In cached mode, the code does not perform those transforms immediately. It packs them into ``geometry_code``, allowing `train.py` to transform groups of samples together after collation.

The low two bits store rotation. Bit 2 stores horizontal flip. Bit 3 stores vertical flip. The operators ``|`` and ``<<`` combine these choices into one integer.

Color-jitter parameters are also deferred in cached mode. In uncached mode, image and mask transformations are applied immediately.

```
geometry_code = (
    rotation
    | (int(flip_horizontal) << 2)
    | (int(flip_vertical) << 3)
)
```

`__getitem__`: returned dictionary

data.py final return block

Each sample is returned as a dictionary. The exact keys depend on whether cache-only information exists.

Required keys include local RGB, context RGB, class mask, Bone/Fibro boundary, Muscle presence, and source name.

Cached samples may also include `'geometry_code'`, `'color_jitter'`, and `'handcrafted'`. The custom collate function in `train.py` knows how to consume those optional keys.

5. train.py - running optimization and saving checkpoints

`'train.py'` is the program coordinator. It does not define the tissue network or image-loading details; it imports those pieces and controls the experiment.

Imports and COARSE_MAPPING

train.py lines 1-23

The imports include command-line parsing, timing, math helpers, DataLoader utilities, the dataset, and the model.

`'COARSE_MAPPING'` translates six final mask IDs into five parent IDs. Fibrocartilage and Muscle both map to parent class one.

deterministic_split_indices

train.py lines 25-41

The function refuses to split fewer than two items.

`'torch.randperm'` creates a random permutation. Supplying a manually seeded generator makes the result repeatable.

The validation count is forced to be at least one and at most one less than the total, ensuring both returned lists are nonempty.

coarse_targets

train.py lines 43-45

The mapping tuple is copied to the mask device. Indexing it with the mask replaces every six-class pixel ID with its five-class parent ID.

multiclass_dice_loss

train.py lines 47-72

The target mask is flattened. `'gather'` selects, for every pixel, the predicted probability assigned to that pixel's correct class.

`'scatter_add_'` accumulates those probabilities into per-class intersections. `'bincount'` counts how many target pixels belong to each class.

The function calculates one Dice loss per class. If no class weights were supplied, it returns the mean; otherwise it normalizes weights and returns a weighted sum.

binary_dice_loss, masked_muscle_loss, and boundary_loss

train.py lines 74-110

Binary Dice is the two-class version used for the boundary map.

The Muscle loss only evaluates pixels whose true label is Fibrocartilage or Muscle. The initial `if` returns a zero-valued differentiable tensor if no valid pixels exist.

Positive weights are calculated from the current batch and clamped to prevent extreme values.

Boundary loss combines weighted binary cross entropy and Dice in a 60/40 mixture.

complete_loss

train.py lines 112-160

Optional six-class weights are converted into matching five-class weights for the coarse head.

The function calculates coarse loss, conditional Muscle loss, six-class Dice, boundary loss, and regional Muscle-presence loss.

The weighted total is the value used for backpropagation. Detached component tensors are retained for reporting without adding more gradient work.

```
total = (
    0.40 * coarse
    + 0.20 * muscle
    + 0.25 * dice
    + 0.10 * boundary_component
    + 0.05 * presence
)
```

Metrics class

train.py lines 162-213

The constructor allocates a confusion matrix and three boundary counters directly on the chosen device.

`update` converts six-class predictions to IDs, encodes each true/predicted pair as one integer, and uses `bincount` to build a confusion matrix efficiently.

Boundary true positives, false positives, and false negatives are accumulated in one tensor.

`results` calculates per-class IoU, mean IoU over valid classes, and boundary F1. Only final results are copied to CPU/Python values.

device_channels_last_collate: sorting and batching

train.py lines 215-248

A collate function receives a Python list of individual samples and returns one batch.

If geometry codes exist, the list is sorted so identical transforms become contiguous. This does not remove randomness because the DataLoader already shuffled sample order.

The inner `tensor\_batch` function extracts one named tensor from every sample and stacks them.

A `while` loop identifies runs of identical geometry codes and stores `(start, end, code)` groups.

device_channels_last_collate: applying geometry

train.py lines 249-303

The inner ``apply_geometry`` function loops over transform groups. Bit tests recover horizontal flip, vertical flip, and rotation from the packed code.

Local and context RGB are concatenated into six channels so each geometric operation is launched once for both images.

Mask and boundary are similarly stacked into two channels and transformed together.

Masks are converted to long integers for cross entropy. Boundaries are converted to float. Muscle presence is computed for the entire batch with one reduction.

Optional handcrafted features and color-jitter parameters are added only when the sample dictionaries contain them.

batched_color_jitter

train.py lines 305-327

The first ``if`` checks batch and parameter shapes before performing arithmetic.

One generator expression creates broadcastable brightness, contrast, saturation, and gamma tensors.

All samples are converted from uint8 and augmented in a few batched operations rather than many per-sample GPU calls.

make_loader

train.py lines 329-352

This helper builds a dictionary of DataLoader options.

Cached CUDA data disables pinned memory and supplies the custom collate function. Uncached CUDA data uses pinned host memory for transfers.

``persistent_workers`` is true only when worker processes exist. ``prefetch_factor`` is also added only when workers are greater than zero.

``drop_last`` keeps training batch shapes consistent when the dataset is at least one full batch.

run_epoch: setup

train.py lines 354-371

``training`` is true when an optimizer was supplied. ``model.train(training)`` switches BatchNorm and other training-sensitive layers between training and evaluation behavior.

Metrics and total-loss tensors are initialized. Gradients are cleared before the loop when training.

run_epoch: the batch loop

train.py lines 372-411

The ``for`` loop visits every batch from the DataLoader.

Required tensors are moved to the selected device. Cached batches are already there, so ``to(device)`` is effectively a no-op.

If color-jitter parameters exist, both RGB inputs are augmented in batches.

Channels-last memory format is enforced on CUDA. Optional handcrafted maps are moved and, when AMP is off, converted to match the model input dtype.

``torch.set_grad_enabled(training)`` avoids building gradients during validation. ``autocast`` uses FP16 where safe when AMP is enabled.

The model returns logits, and ``complete_loss`` turns them into one total loss plus metrics probabilities.

run_epoch: backward pass and accumulation*train.py lines 412-426*

The loss is divided by `accumulation_steps` before backpropagation so accumulated gradients have the intended scale.

`should_step` becomes true after the requested number of batches or on the final incomplete accumulation group.

Before stepping, gradients are unscaled and clipped. The scaler performs the optimizer step, updates its scale, and gradients are cleared.

Key distinction: A batch performs one forward/backward pass. An optimizer update happens only when `scaler.step(optimizer)` runs. Gradient accumulation can therefore reduce updates per epoch.

run_epoch: accumulation of reports*train.py lines 427-447*

Loss components and metrics are accumulated after each batch.

At the end, mean values are calculated and copied to CPU/Python only once. This avoids the frequent GPU synchronization caused by calling `.item()` every batch.

parse_args: command-line interface*train.py lines 449-553*

The parser declares every option accepted after `python train.py`.

Options without `action="store_true"` expect a value. Flags such as `--compile-model` become Boolean true when present.

Related groups control data paths, caches, patch splitting, augmentation probabilities, learning rates, performance features, outputs, resume, and random seed.

parse_args: validation if statements*train.py lines 554-577*

These checks fail early with readable command-line errors.

They enforce legal crop size, context scale, batch and accumulation sizes, validation fraction, patch-pool size, probability sum, nonnegative VRAM reserve, cache dependency, and minimum feature tile size.

main: seeds, device selection, AMP, and TF32*train.py lines 579-613*

Seeds are set for PyTorch, Python random, and NumPy so deterministic split choices and many sampling choices are repeatable.

If CUDA was explicitly requested but is unavailable, training stops.

The nested conditional expression chooses CUDA for explicit CUDA or auto-with-CUDA; otherwise CPU.

CUDA enables cuDNN benchmarking, optional TF32, and AMP. CPU mode sets a thread count and rejects model compilation in this implementation.

main: resume configuration*train.py lines 614-624*

If `--resume` was supplied, the checkpoint is loaded into CPU memory first.

The runtime context scale is read from the checkpoint configuration so resumed data preparation matches the saved model.

main: choosing the validation split

train.py lines 625-677

When `--val-data` is supplied, the two directory trees are used directly.

Without it, the default patch-level split creates a deterministic patch pool and assigns patch IDs to training or validation. The warning explains that this is not an independent specimen test.

The alternative image-level branch splits source-image IDs and requires at least two images.

main: VRAM cache check

train.py lines 678-734

This block runs only when CUDA image caching is requested.

It determines the exact training image list, estimates each cache component, asks CUDA how much memory is free, subtracts the requested reserve, and prints a breakdown.

If optional structural features are cached, a note explains the color-jitter tradeoff.

If the estimated cache exceeds usable memory, training stops before partial allocation.

main: dataset and loader construction

train.py lines 735-811

Patch-level splitting disables class/boundary-focused sampling and gives both base datasets the same deterministic patch pool.

`Subset` wraps the base datasets with the assigned patch IDs.

Cached CUDA training forces zero workers and uses the device collate function. Validation remains an ordinary DataLoader.

main: workload report and update warning

train.py lines 812-852

The data split is stored in a dictionary for checkpoints.

Optimizer updates are estimated as the number of batches divided by accumulation steps, rounded up.

When fewer than 25 updates occur per epoch, the program prints the large-batch warning. This is advisory, not an exception.

main: model and parameter groups

train.py lines 853-886

A resumed run uses the checkpoint model configuration; a new run builds one from current arguments.

The model is moved into channels-last format and then to the device.

The `for name, parameter in model.named\_parameters()` loop separates pretrained encoder parameters from newer parameters by name prefix. The two groups receive different learning rates.

AdamW uses the fused CUDA implementation unless disabled.

main: scheduler, scaler, weights, and resume state

train.py lines 887-905

ReduceLROnPlateau watches validation mIoU. If it fails to improve for six epochs, learning rates are multiplied by 0.5, down to a minimum.

GradScaler supports AMP. Optional class weights are copied to the device.

A resumed checkpoint restores model, optimizer, scheduler, scaler, epoch, and best score.

main: torch.compile

train.py lines 906-911

When requested, `model.compile(mode=...)` asks PyTorch to capture and optimize repeated model operations.

Compilation happens during the first actual steps, so startup is slower. The message prepares the user for that delay.

main: epoch loop and speed measurement

train.py lines 912-940

Output directories are created if needed.

The `for epoch` loop runs from the starting epoch through the requested final epoch, inclusive.

CUDA is synchronized before and after the training epoch so wall-clock timing includes completed GPU work. Patches per second is dataset length divided by training seconds.

Validation runs under `torch.inference_mode()`, which disables gradients and related overhead.

main: logging, checkpoint creation, and best-model if statement

train.py lines 941-980

Validation mIoU is passed to the scheduler. Class IoUs and loss components are formatted into readable text.

The checkpoint dictionary stores model weights, configuration, epoch, metrics, optimizer, scheduler, scaler, command-line arguments, and data-split IDs.

`last_model.pt` is saved every completed epoch and overwritten.

The final `if val_iou > best_iou` saves `best_model.pt` only when validation mIoU is strictly better than every earlier completed epoch.

Ctrl+C behavior: There is no `try/except KeyboardInterrupt` around the epoch loop in the reviewed revision. Interrupting during an epoch does not save the partially completed epoch. The previously completed `last_model.pt` remains available.

The main guard

train.py lines 983-984

Running `python train.py ...` reaches this block and calls `main()`.

Importing a helper from `train.py` does not start training because the condition is false during import.

6. Utility scripts and cluster files

The detailed block-by-block treatment above covers the core training pipeline. The remaining files are shorter entry points around that core.

predict.py

- Loads a saved checkpoint and recreates AccurateTissueNet from ``model_config``.
- Opens one native-resolution image and covers it with overlapping tiles.
- Builds matching wider context inputs, runs batches through the model, and blends overlapping predictions with center-weighted windows.
- Writes hard class IDs, a color mask, per-class grayscale probability PNGs, a boundary image, and optionally the full probability array.
- Uses inference-only modes; it does not calculate loss or change model weights.

quantify.py

- Reads the prediction directory and calibrated pixel dimensions.
- Counts hard-mask pixels per class and sums probability mass per class.
- Converts pixel area from square micrometers to square millimeters.
- Writes an area-summary CSV for downstream validation against expert measurements.

smoke_test.py

- Builds the model and synthetic input tensors.
- Checks imports, output shapes, probability sums, and optionally backward propagation.
- Provides a fast environment check before committing to a long training run.

SLURM scripts

- Shell files beginning with ``#SBATCH`` request cluster resources before launching the same Python scripts.
- Environment variables supply data paths, virtual environment, outputs, cache options, and extra arguments.
- They do not define a different model; they automate how the existing Python programs are submitted to a scheduler.

7. Reading errors and making safe changes

Tracebacks are read from the bottom upward

The last line usually states the immediate error. The preceding ``File ... line ...`` entries show the chain of function calls that led there. Start at the bottom, then move upward until you reach your own project file.

Warnings versus exceptions

- A warning such as the ``max_autotune_gemm`` SM message reports a skipped optimization; training can continue.
- The “fewer than 25 optimizer updates” message is a project advisory; it does not stop training.
- ``raise ValueError`` and ``raise RuntimeError`` stop the program unless caught.

Changes that require retraining

- Changing class order or the meaning of mask IDs.
- Changing encoder structure, decoder widths, head structure, or feature-stem channel counts.
- Changing how probabilities are assembled from parent and child heads.

Changes that usually preserve checkpoint compatibility

- Improving data-loading efficiency without changing returned values.
- Changing logging frequency or adding timing output.
- Adding safer checkpoint handling around KeyboardInterrupt.
- Changing DataLoader worker count, cache choice, compile mode, or fused optimizer choice.

A practical debugging sequence

9. Run ``python smoke_test.py --device cuda``.
10. Run one epoch with a small batch and compilation off.
11. Verify shapes, class IDs, and checkpoint creation.
12. Enable the image cache and confirm the printed VRAM estimate.
13. Enable compilation and ignore the first-step timing.
14. Compare steady-state patches/s and validation metrics rather than GPU percentage alone.

8. Glossary

Term	Plain-language meaning
Batch	Several patches processed together in one forward/backward pass.
Checkpoint	A file containing model weights and usually optimizer, scheduler, scaler, epoch, and configuration state.
Collate function	The function that combines individual dataset samples into one batch.
Context crop	A wider physical field of view resized to the same tensor dimensions as the local crop.
Epoch	One complete pass through the dataset object as currently defined.
Forward pass	Sending inputs through the model to calculate outputs.
Gradient	Derivative of loss with respect to a trainable parameter.
Logit	Unnormalized model output before softmax or sigmoid.
Optimizer update	One call that actually changes model parameters using accumulated gradients.
Patch	One local crop, its context crop, and aligned target data.
Residual connection	A path that adds an earlier tensor to a transformed tensor.
Tensor	A multidimensional numerical array used by

	PyTorch.
VRAM cache	Source data stored in GPU memory to avoid repeated disk reads and host-to-device transfers.

Recommended reading order: Read `model.py` first to understand what the network expects, then `data.py` to see how those inputs are created, and finally `train.py` to see how the pieces are repeatedly executed and optimized.

Source basis

This guide was prepared from the public CNN-MalGen06 master branch reviewed on July 31, 2026, especially the current `data.py`, `model.py`, `train.py`, and `README.md`. Utility-script descriptions follow the repository's documented behavior. If the repository changes, compare the new functions and line ranges before relying on this guide as an exact reference.